Университет ИТМО
Факультет программной инженерии и компьютерной техники

Курсовая работа

Этап 2. Модель данных и её реализация в PostgreSQL

Информационная система «Бдительность»

Выполнил: студент группы P3310
Мыц Иван Евгеньевич

Санкт-Петербург, 2026

1. ER-модель

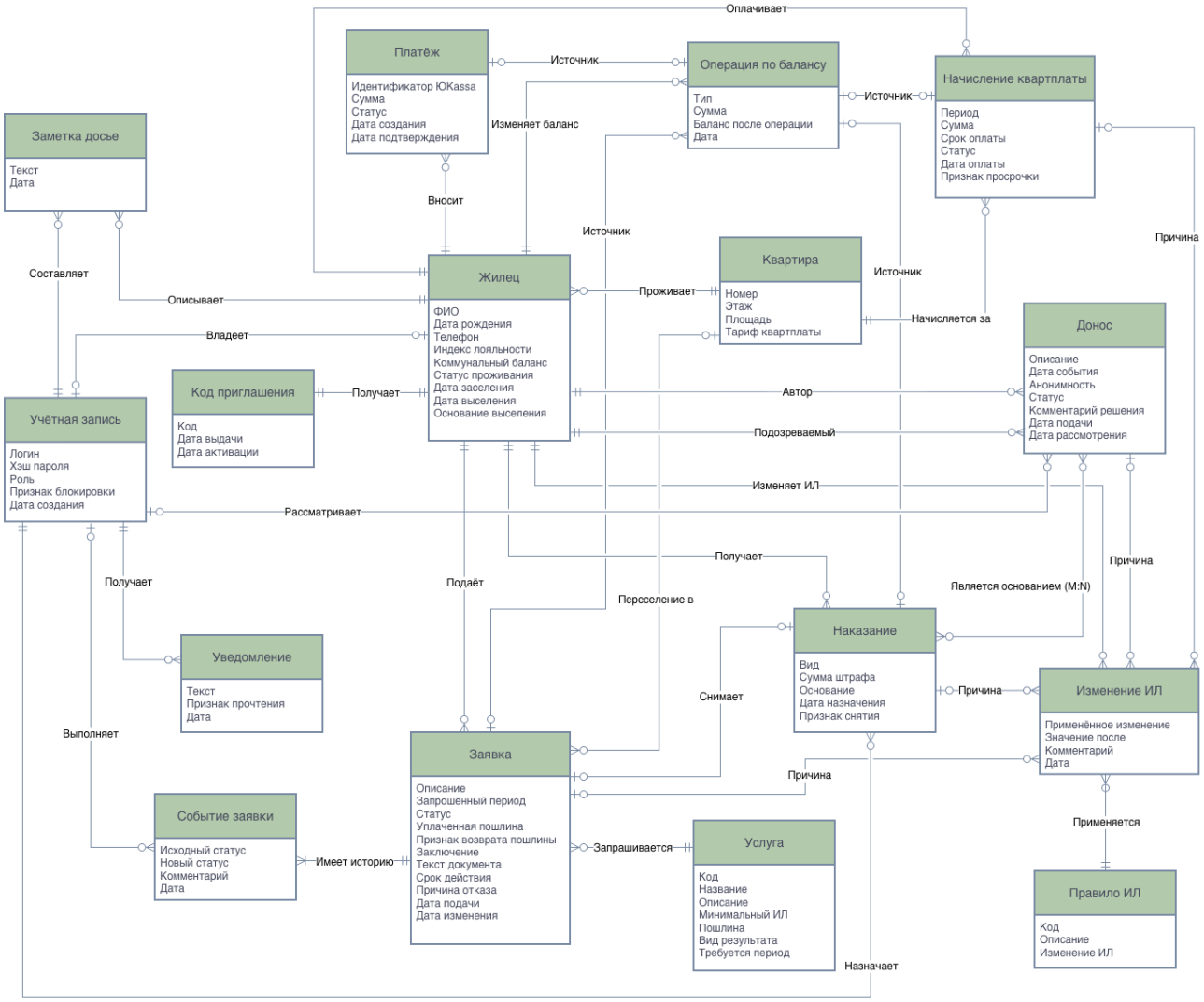


Таблица 1 – Сущности

Сущность	Описание	Атрибуты
Учётная запись	Данные для входа жильца или управляющего.	логин, хеш пароля, роль, признак блокировки, дата создания
Жилец	Человек, проживающий в доме.	ФИО, дата рождения, телефон, индекс лояльности, коммунальный баланс, статус проживания, даты заселения и выселения, основание выселения
Код приглашения	Одноразовый код для активации учётной записи.	код, дата выдачи, дата активации
Квартира	Квартира дома, в ней живёт до 10 жильцов.	номер, этаж, площадь, тариф квартплаты
Услуга	Услуга из фиксированного перечня SV01–SV09.	код, название, описание, минимальный ИЛ, пошлина, вид результата, требуется период

Сущность	Описание	Атрибуты
Заявка	Обращение жильца за услугой и его результат.	описание, запрошенный период, статус, уплаченная пошлина, признак возврата пошлины, заключение, текст документа, срок действия, причина отказа, даты подачи и изменения
Событие заявки	Запись истории обработки заявки.	исходный статус, новый статус, комментарий, дата
Донос	Сообщение жильца о нарушении соседа.	описание, дата события, анонимность, статус, комментарий решения, даты подачи и рассмотрения
Наказание	Предупреждение, штраф или выселение.	вид, сумма штрафа, основание, дата назначения, признак снятия
Платёж	Пополнение баланса через ЮKassa.	идентификатор ЮKassa, сумма, статус, даты создания и подтверждения
Начисление квартплаты	Ежемесячное начисление за квартиру; оплачивает любой её жилец.	период, сумма, срок оплаты, статус, дата оплаты, признак просрочки
Операция по балансу	Запись журнала изменений баланса.	тип, сумма, баланс после операции, дата
Правило ИЛ	Правило изменения ИЛ RL01–RL07.	код, описание, изменение ИЛ
Изменение ИЛ	Запись журнала изменений ИЛ.	применённое изменение, значение после, комментарий, дата
Заметка досье	Наблюдение управляющего о жильце.	текст, дата
Уведомление	Сообщение пользователю о событии.	текст, признак прочтения, дата

Таблица 2 – Связи

Связь	Тип связи	Пояснение
Учётная запись — Жилец	0..1 : 0..1	Учётная запись появляется после активации кода; у управляющего записи жильца нет.
Учётная запись — Уведомление	1 : 0..*	Пользователь получает уведомления.
Учётная запись — Заметка досье	1 : 0..*	Управляющий составляет заметку.
Жилец — Заметка досье	1 : 0..*	Заметка описывает жильца.
Жилец — Код приглашения	1 : 1	При заселении жильцу выдаётся код.
Жилец — Квартира	0..* : 1	Жилец проживает в квартире; одновременно в квартире живут до 10 жильцов.
Жилец — Донос	1 : 0..*	Жилец — автор доноса.
Жилец — Донос	1 : 0..*	Жилец — подозреваемый по доносу.
Учётная запись — Донос	0..1 : 0..*	Управляющий рассматривает донос.
Жилец — Заявка	1 : 0..*	Жилец подаёт заявки.
Услуга — Заявка	1 : 0..*	Заявка подаётся на одну услугу.

Связь	Тип связи	Пояснение
Квартира — Заявка	0..1 : 0..*	Квартира для переселения (SV07).
Заявка — Событие заявки	1 : 1..*	Заявка имеет историю обработки.
Учётная запись — Событие заявки	0..1 : 0..*	Пользователь, изменивший статус.
Жилец — Наказание	1 : 0..*	Жилец получает наказание.
Учётная запись — Наказание	1 : 0..*	Управляющий назначает наказание.
Донос — Наказание	**0..* : 0..***	**Наказание назначается по одному или нескольким доносам; по доносу может быть назначено несколько наказаний.**
Заявка — Наказание	0..1 : 0..1	Заявка SV09 снимает предупреждение.
Жилец — Платёж	1 : 0..*	Жилец вносит платёж.
Жилец — Начисление квартплаты	0..1 : 0..*	Начисление оплачивает один из жильцов квартиры.
Квартира — Начисление квартплаты	1 : 0..*	Квартплата начисляется за квартиру.
Жилец — Операция по балансу	1 : 0..*	Операция изменяет баланс жильца.
Платёж, Начисление, Заявка, Наказание — Операция по балансу	0..1 : 0..1	Источник операции (для заявки 0..1 : 0..*: пошлина и её возврат).
Жилец — Изменение ИЛ	1 : 0..*	Изменение ИЛ жильца.
Правило ИЛ — Изменение ИЛ	1 : 0..*	Изменение выполняется по правилу.
Донос, Наказание, Начисление, Заявка — Изменение ИЛ	0..1 : 0..*	Причина изменения ИЛ.

УДАЛИТЬ Связь «многие-ко-многим» здесь одна — «Донос — Наказание». Пример из тестовых данных: штраф Мельнику назначен сразу по трём доносам, а Ершову по одному доносу назначены и штраф, и предупреждение. «Учётная запись — Жилец» необязательна с обеих сторон: жилец существует до активации кода, а у управляющего есть учётная запись, но нет записи жильца. «Заявка — Наказание»: выполнение услуги SV09 «Снятие предупреждения» помечает последнее действующее предупреждение снятым, само наказание не удаляется. «Операция по балансу» и «Изменение ИЛ» — журналы: баланс и ИЛ жильца всегда равны сумме записей журнала, записи не изменяются и не удаляются. **УДАЛИТЬ**

2. Даталогическая модель

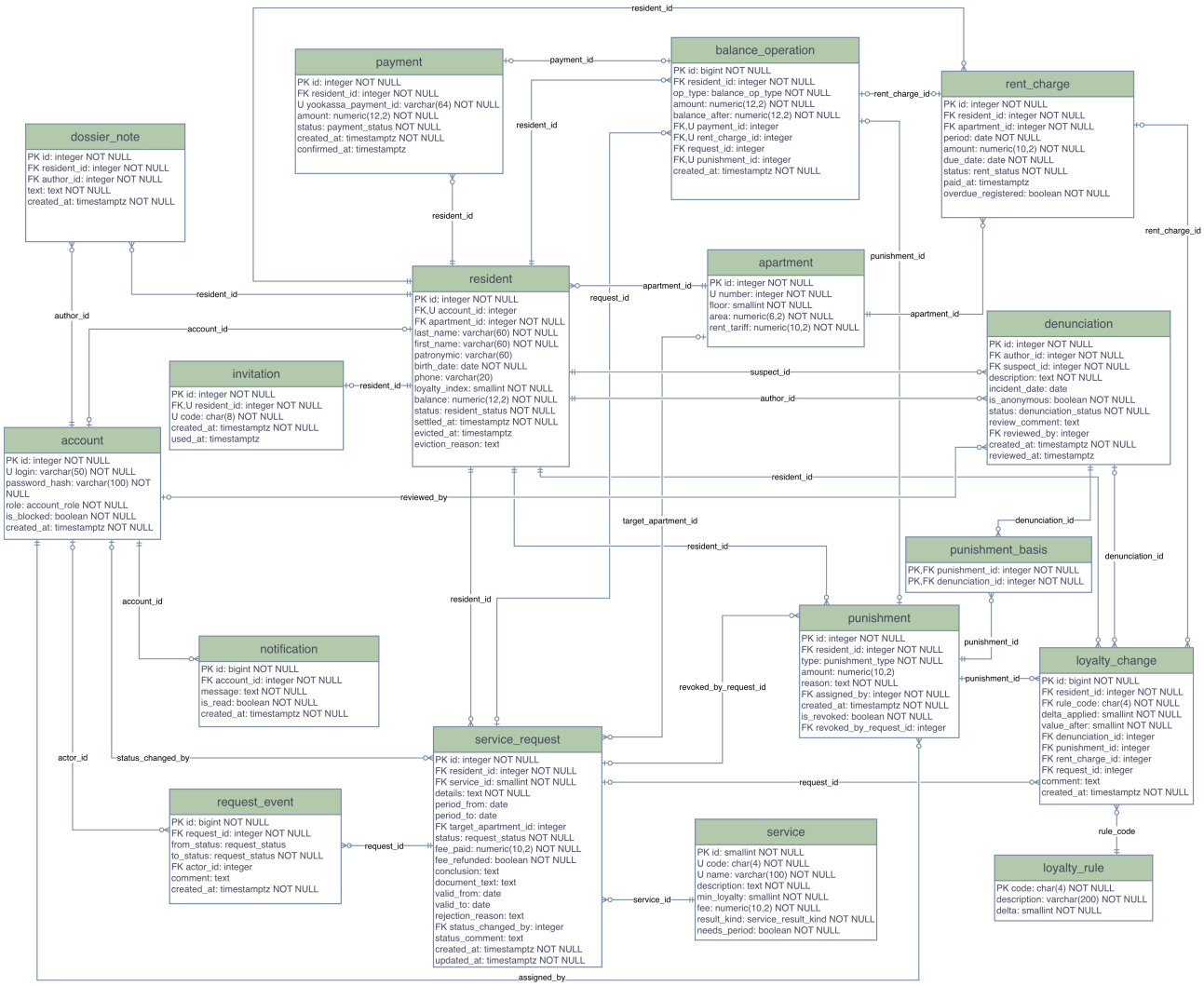


Таблица 3 – Первичные и внешние ключи

Таблица	Сущность	Первичный ключ	Внешние ключи
account	Учётная запись	id	—
resident	Жилец	id	account_id (U), apartment_id
invitation	Код приглашения	id	resident_id (U)
apartment	Квартира	id	—
service	Услуга	id	—
service_request	Заявка	id	resident_id, service_id, target_apartment_id, status_changed_by
request_event	Событие заявки	id	request_id, actor_id
denunciation	Донос	id	author_id, suspect_id, reviewed_by
punishment	Наказание	id	resident_id, assigned_by, revoked_by_request_id

Таблица	Сущность	Первичный ключ	Внешние ключи
punishment_basis	Связь M:N «Донос — Наказание»	(punishment_id, denunciation_id)	punishment_id, denunciation_id
payment	Платёж	id	resident_id
rent_charge	Начисление квартплаты	id	apartment_id, paid_by
balance_operation	Операция по балансу	id	resident_id, payment_id (U), rent_charge_id (U), request_id, punishment_id (U)
loyalty_rule	Правило ИЛ	code	—
loyalty_change	Изменение ИЛ	id	resident_id, rule_code, denunciation_id, punishment_id, rent_charge_id, request_id
dossier_note	Заметка досье	id	resident_id, author_id
notification	Уведомление	id	account_id

Все таблицы находятся в третьей нормальной форме: значения атрибутов атомарны, каждый неключевой атрибут зависит от всего первичного ключа и не зависит от других неключевых атрибутов.

УДАЛИТЬ Если спросят про `resident.balance` и `resident.loyalty_index`: это не нарушение 3НФ. Нормальные формы описывают зависимости внутри одной таблицы, а эти столбцы зависят только от `id` жильца. Они совпадают с суммами журналов `balance_operation` и `loyalty_change` и хранятся, чтобы быстро проверять условия подачи заявок и сортировать очередь. Совпадение гарантируют триггеры: напрямую изменить баланс или ИЛ нельзя. `service_request.fee_paid` и `rent_charge.amount` — не копии `service.fee` и `apartment.rent_tariff`, а значения на момент операции: тариф может измениться, а история — нет. Отрицательный баланс означает задолженность по штрафам, поэтому отдельного поля для долга нет. **УДАЛИТЬ**

3. Реализация в PostgreSQL

Модель реализована набором SQL-скриптов (таблица 4).

Таблица 4 – Скрипты

Файл	Назначение
install.sql	Создание базы целиком: выполняет скрипты 01–04.
00_drop.sql	Удаление всех объектов базы.
01_create.sql	Типы, таблицы, ограничения, справочники услуг и правил ИЛ.
02_triggers.sql	Триггеры.
03_functions.sql	Функции для критически важных запросов и представления.

Файл	Назначение
04_indexes.sql	Индексы.
05_fill.sql	Тестовые данные: 30 квартир, 24 жильца, заявки во всех статусах, доносы, наказания.
06_fill_bulk.sql	Массовые данные для проверки индексов: 300 жильцов, 10 000 заявок и 10 000 доносов.
tests/	Тесты базы на Java (JUnit 5, JDBC): 14 тестов в двух классах, запуск extttmvn test.

```
psql -h pg -d studs -f install.sql
psql -h pg -d studs -f 05_fill.sql
cd tests && mvn test
```

УДАЛИТЬ Тесты проверяют ограничения таблиц, журналы баланса и ИЛ, вместимость квартиры, модели состояний заявки и доноса, связь M:N и каждую функцию. Каждый тест сам создаёт нужные строки в своей транзакции и откатывает её, поэтому тесты проходят и на пустой, и на заполненной базе. Ошибки проверяются по коду SQLSTATE. **УДАЛИТЬ**

УДАЛИТЬ На helios нельзя выполнить CREATE DATABASE, поэтому скрипты создают объекты в личной схеме базы studs. Нужен PostgreSQL 11 или новее. Целостность обеспечена ограничениями NOT NULL, CHECK, UNIQUE, внешними ключами и 10 триггерами — их описание в файле с вопросами к защите и в листинге 02_triggers.sql. Триггеры защищают только данные: журналы, согласованность баланса и ИЛ с журналами, вместимость квартиры (до 10 жильцов), модели состояний и историю заявок. Пароли тестовых пользователей: manager/manager123, жильцы /resident123. **УДАЛИТЬ**

4. Функции

Бизнес-логику реализует серверная часть на Spring, простые чтения и изменения она выполняет через ORM. Функции предложены для критически важных запросов: операций, которые должны выполняться атомарно, и выборок, которые удобнее выполнить одним запросом (таблица 5). Представление v_denunciation_for_manager скрывает автора анонимного доноса, v_loyalty_history — ссылки на доносы в истории ИЛ.

Таблица 5 – Функции

Имя	Где используется	Что делает
activate_account	UC01	Помечает код приглашения использованным, создаёт учётную запись и привязывает её к жильцу.
apply_payment_status	UC04	Применяет статус платежа из ЮKassa и зачисляет деньги ровно один раз.
pay_rent	UC06	Отмечает начисление квартиры оплаченным жильцом и списывает сумму с его баланса; повторно оплатить нельзя.
accrue_rent	S001	Начисляет квартплату за месяц одним запросом: одно начисление на квартиру с жильцами.
register_overdue_rent	S002	Отмечает просроченные начисления и возвращает жильцов этих квартир.

Имя	Где используется	Что делает
request_queue	UC17	Возвращает очередь необработанных заявок по убыванию ИЛ заявителей.
resident_dossier	UC15	Возвращает досье жильца одним документом JSON без авторов анонимных доносов.

УДАЛИТЬ Почему именно эти функции: активация, зачисление платежа и оплата квартплаты меняют несколько таблиц и должны выполняться целиком или не выполняться вовсе; повторный вызов apply_payment_status ничего не меняет, поэтому опрос ЮKassa безопасен. Начисление и просрочка квартплаты обрабатывают всех жильцов одним запросом. Очередь заявок и досье — выборки из многих таблиц, которые удобно получить одним вызовом. Остальные прецеденты — обычные чтения и изменения через ORM: условия подачи заявок, правила ИЛ, уведомления, последствия наказаний и права пользователей проверяет Spring. Ошибки имеют коды SQLSTATE: ВН404 — не найдено, ВН409 — недопустимо в текущем состоянии, ВН422 — не выполнено условие; Spring по коду выбирает ответ HTTP. **УДАЛИТЬ**

5. Индексы

Индексы созданы для запросов, которые выполняются при каждом открытии страниц кабинета (таблица 6). Индексы первичных ключей и ограничений UNIQUE PostgreSQL создаёт сам.

Таблица 6 – Индексы

№	Индекс	Столбцы	UC	Где используется
I1	idx_request_active	service_request (status, created_at), только необработанные заявки	UC17	Очередь заявок (request_queue)
I2	idx_request_resident	service_request (resident_id, created_at)	UC10	Список заявок жильца
I3	idx_request_event_request	request_event (request_id, created_at)	UC10	История обработки заявки
I4	idx_denunciation_pending	denunciation (created_at), только нерассмотренные доносы	UC22	Очередь доносов
I5	idx_balance_operation_resident	balance_operation (resident_id, created_at)	UC05	История операций по балансу
I6	idx_notification_unread	notification (account_id, created_at), только непрочитанные	U019	Непрочитанные уведомления

УДАЛИТЬ Индексы I1, I4 и I6 частичные (WHERE): в них попадают только «активные»

строки, поэтому они маленькие и не растут вместе с историей. Индексы по внешним ключам PostgreSQL автоматически не создаёт, поэтому без I2, I3 и I5 запрос «все записи жильца» читает всю таблицу. Запросы для I2–I6 приложение выполняет через ORM, индекс используется независимо от того, кто сформировал запрос. **УДАЛИТЬ**

5.1. Проверка

Индексы проверены на массовых данных (`06_fill_bulk.sql`: 300 жильцов, 10 013 заявок, 28 434 записей истории заявок, 10 023 доносов, 18 568 уведомлений). Для каждого запроса выполнена команда `EXPLAIN ANALYZE` без индексов и с ними (таблица 7).

Таблица 7 – Время выполнения запросов

Запрос	**Инд.**	**Без индекса**	**мс**	**С индексом**	**мс**	**Уск.**
Очередь заявок	I1	Seq Scan	2.73	Bitmap Index Scan	1.57	1.7 ×
Заявки жильца	I2	Seq Scan	1.11	Index Scan	0.11	10 ×
История заявки	I3	Seq Scan	1.41	Bitmap Index Scan	0.13	11 ×
Очередь доносов	I4	Seq Scan	0.90	Index Scan	0.07	14 ×
Операции по балансу	I5	Seq Scan	0.44	Bitmap Index Scan	0.17	3 ×
Непрочитанные уведомления	I6	Seq Scan	1.21	Index Scan	0.07	16 ×

Без индексов каждый запрос читает всю таблицу (`Seq Scan`), с индексами — только нужные строки. Сильнее всего ускорились выборки нескольких строк из больших таблиц: история заявки, непрочитанные уведомления, заявки жильца.

УДАЛИТЬ Очередь заявок (I1) ускоряется слабо: в результат попадают все необработанные заявки, и они сортируются по ИЛ заявителя из другой таблицы. Индекс полезен тем, что объём чтения не зависит от числа уже выполненных заявок, которые копятся годами. **УДАЛИТЬ**

6. Заключение

Построены ER-модель из 16 сущностей со связью «многие-ко-многим» и даталогическая модель из 17 таблиц. Модель реализована в PostgreSQL вместе с ограничениями целостности, триггерами, функциями для критически важных запросов и индексами; написаны скрипты создания, удаления и заполнения базы.

Приложение А. Листинги скриптов

01_create.sql

```
-- =================================================================
-- ИС Бдительность«». Этап 2.
-- 01_create.sql — типы, таблицы, ограничения и справочные данные.
-- Объекты создаются в текущей схеме на( helios — в личной схеме базы studs).
-- =================================================================

BEGIN;

-- Перечислимые типы
CREATE TYPE account_role        AS ENUM ('RESIDENT', 'MANAGER');
CREATE TYPE resident_status     AS ENUM ('ACTIVE', 'EVICTED');
CREATE TYPE service_result_kind AS ENUM ('WORK_MARK', 'DOCUMENT', 'TICKET', 'PERMIT', 'RELOCATION',
    'WARNING_REMOVAL');
CREATE TYPE request_status      AS ENUM ('SUBMITTED', 'IN_PROGRESS', 'NEEDS_CLARIFICATION', '
    COMPLETED', 'REJECTED', 'CANCELLED');
CREATE TYPE denunciation_status AS ENUM ('SUBMITTED', 'CONFIRMED', 'REJECTED', 'FALSE');
CREATE TYPE punishment_type     AS ENUM ('WARNING', 'FINE', 'EVICTION');
CREATE TYPE payment_status      AS ENUM ('PENDING', 'SUCCEEDED', 'CANCELED');
CREATE TYPE rent_status         AS ENUM ('UNPAID', 'PAID');
CREATE TYPE balance_op_type     AS ENUM ('TOPUP', 'RENT', 'FEE', 'FEE_REFUND', 'FINE');

-- Учётная запись
CREATE TABLE account (
    id             SERIAL PRIMARY KEY,
    login          VARCHAR(50)  NOT NULL UNIQUE CHECK (login ~ '^[A-Za-z0-9_.]{3,50}$'),
    password_hash  VARCHAR(100) NOT NULL,
    role           account_role NOT NULL,
    is_blocked     BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT now()
);
-- управляющий в системе один ограничение( 2 SRS)
CREATE UNIQUE INDEX uq_account_single_manager ON account (role) WHERE role = 'MANAGER';

-- Квартира
CREATE TABLE apartment (
    id           SERIAL PRIMARY KEY,
    number       INTEGER       NOT NULL UNIQUE CHECK (number > 0),
    floor        SMALLINT      NOT NULL CHECK (floor BETWEEN 1 AND 50),
    area         NUMERIC(6,2)  NOT NULL CHECK (area > 0),
    rent_tariff  NUMERIC(10,2) NOT NULL CHECK (rent_tariff >= 0)
);

-- Жилец в( квартире не более 10 жильцов — проверяет триггер resident_guard;
-- отрицательный баланс — задолженность по штрафам)
CREATE TABLE resident (
    id              SERIAL PRIMARY KEY,
    account_id      INTEGER UNIQUE REFERENCES account (id),
    apartment_id    INTEGER NOT NULL REFERENCES apartment (id),
    last_name       VARCHAR(60)     NOT NULL CHECK (btrim(last_name)  <> ''),
    first_name      VARCHAR(60)     NOT NULL CHECK (btrim(first_name) <> ''),
    patronymic      VARCHAR(60),
    birth_date      DATE            NOT NULL CHECK (birth_date > DATE '1900-01-01'),
    phone           VARCHAR(20)     CHECK (phone ~ '^\+?[0-9\-␣]{5,20}$'),
    loyalty_index   SMALLINT        NOT NULL DEFAULT 50 CHECK (loyalty_index BETWEEN 0 AND 100),
    balance         NUMERIC(12,2)   NOT NULL DEFAULT 0,
    status          resident_status NOT NULL DEFAULT 'ACTIVE',
    settled_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    evicted_at      TIMESTAMPTZ,
    eviction_reason TEXT,
    CHECK ((status = 'EVICTED') = (evicted_at IS NOT NULL)),
    CHECK (status = 'ACTIVE' OR eviction_reason IS NOT NULL)
);

-- Код приглашения один( на жильца, используется один раз)
CREATE TABLE invitation (
    id          SERIAL PRIMARY KEY,
    resident_id INTEGER     NOT NULL UNIQUE REFERENCES resident (id) ON DELETE CASCADE,
    code        CHAR(8)     NOT NULL UNIQUE CHECK (code ~ '^[A-Z0-9]{8}$'),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    used_at     TIMESTAMPTZ CHECK (used_at >= created_at)
);
```

```
-- Услуга фиксированный( перечень –SV01SV09)
CREATE TABLE service (
    id            SMALLSERIAL PRIMARY KEY,
    code          CHAR(4)             NOT NULL UNIQUE CHECK (code ~ '^SV[0–9]{2}$'),
    name          VARCHAR(100)        NOT NULL UNIQUE,
    description   TEXT                NOT NULL,
    min_loyalty   SMALLINT            NOT NULL CHECK (min_loyalty BETWEEN 0 AND 100),
    fee           NUMERIC(10,2)       NOT NULL CHECK (fee >= 0),
    result_kind   service_result_kind NOT NULL,
    needs_period  BOOLEAN             NOT NULL DEFAULT FALSE
);

-- Заявка на услугу. status_changed_by и status_comment — кто и с каким
-- комментарием изменил статус последним; триггер переносит их в историю.
CREATE TABLE service_request (
    id                  SERIAL PRIMARY KEY,
    resident_id         INTEGER        NOT NULL REFERENCES resident (id),
    service_id          SMALLINT       NOT NULL REFERENCES service (id),
    details             TEXT           NOT NULL CHECK (btrim(details) <> ''),
    period_from         DATE,
    period_to           DATE,
    target_apartment_id INTEGER        REFERENCES apartment (id),
    status              request_status NOT NULL DEFAULT 'SUBMITTED',
    fee_paid            NUMERIC(10,2)  NOT NULL CHECK (fee_paid >= 0),
    fee_refunded        BOOLEAN        NOT NULL DEFAULT FALSE,
    conclusion          TEXT,
    document_text       TEXT,
    valid_from          DATE,
    valid_to            DATE,
    rejection_reason    TEXT,
    status_changed_by   INTEGER        REFERENCES account (id),
    status_comment      TEXT,
    created_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CHECK (period_from <= period_to),
    CHECK (valid_from <= valid_to),
    CHECK (status <> 'COMPLETED' OR coalesce(btrim(conclusion), '') <> ''),
    CHECK (status <> 'REJECTED'  OR coalesce(btrim(rejection_reason), '') <> ''),
    CHECK (NOT fee_refunded OR status IN ('REJECTED', 'CANCELLED'))
);

-- История обработки заявки журнал()
CREATE TABLE request_event (
    id          BIGSERIAL PRIMARY KEY,
    request_id  INTEGER        NOT NULL REFERENCES service_request (id),
    from_status request_status,
    to_status   request_status NOT NULL,
    actor_id    INTEGER        REFERENCES account (id),
    comment     TEXT,
    created_at  TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CHECK (from_status IS DISTINCT FROM to_status)
);

-- Донос
CREATE TABLE denunciation (
    id              SERIAL PRIMARY KEY,
    author_id       INTEGER             NOT NULL REFERENCES resident (id),
    suspect_id      INTEGER             NOT NULL REFERENCES resident (id),
    description     TEXT                NOT NULL CHECK (length(btrim(description)) >= 10),
    incident_date   DATE,
    is_anonymous    BOOLEAN             NOT NULL DEFAULT FALSE,
    status          denunciation_status NOT NULL DEFAULT 'SUBMITTED',
    review_comment  TEXT,
    reviewed_by     INTEGER             REFERENCES account (id),
    created_at      TIMESTAMPTZ         NOT NULL DEFAULT now(),
    reviewed_at     TIMESTAMPTZ,
    CHECK (author_id <> suspect_id),
    CHECK ((status = 'SUBMITTED') = (reviewed_at IS NULL)),
    CHECK (reviewed_at >= created_at)
);

-- Наказание сумма( — только у штрафа; снять можно только предупреждение)
CREATE TABLE punishment (
    id                  SERIAL PRIMARY KEY,
    resident_id         INTEGER         NOT NULL REFERENCES resident (id),
```

```
    type                  punishment_type NOT NULL,
    amount                NUMERIC(10,2)   CHECK (amount > 0),
    reason                TEXT            NOT NULL CHECK (btrim(reason) <> ''),
    assigned_by           INTEGER         NOT NULL REFERENCES account (id),
    created_at            TIMESTAMPTZ     NOT NULL DEFAULT now(),
    is_revoked            BOOLEAN         NOT NULL DEFAULT FALSE,
    revoked_by_request_id INTEGER         REFERENCES service_request (id),
    CHECK ((type = 'FINE') = (amount IS NOT NULL)),
    CHECK (NOT is_revoked OR (type = 'WARNING' AND revoked_by_request_id IS NOT NULL))
);

-- Основание наказания: связь M:N наказание« — донос»
CREATE TABLE punishment_basis (
    punishment_id    INTEGER NOT NULL REFERENCES punishment (id) ON DELETE CASCADE,
    denunciation_id  INTEGER NOT NULL REFERENCES denunciation (id),
    PRIMARY KEY (punishment_id, denunciation_id)
);

-- Платёж ЮKassa
CREATE TABLE payment (
    id                  SERIAL PRIMARY KEY,
    resident_id         INTEGER        NOT NULL REFERENCES resident (id),
    yookassa_payment_id VARCHAR(64)    NOT NULL UNIQUE,
    amount              NUMERIC(12,2)  NOT NULL CHECK (amount > 0),
    status              payment_status NOT NULL DEFAULT 'PENDING',
    created_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    confirmed_at        TIMESTAMPTZ,
    CHECK ((status = 'SUCCEEDED') = (confirmed_at IS NOT NULL))
);

-- Начисление квартплаты: одно на квартиру за месяц, оплачивает любой её жилец
CREATE TABLE rent_charge (
    id                 SERIAL PRIMARY KEY,
    apartment_id       INTEGER       NOT NULL REFERENCES apartment (id),
    period             DATE          NOT NULL CHECK (extract(day FROM period) = 1),
    amount             NUMERIC(10,2) NOT NULL CHECK (amount >= 0),
    due_date           DATE          NOT NULL,
    status             rent_status   NOT NULL DEFAULT 'UNPAID',
    paid_at            TIMESTAMPTZ,
    paid_by            INTEGER       REFERENCES resident (id),
    overdue_registered BOOLEAN       NOT NULL DEFAULT FALSE,
    UNIQUE (apartment_id, period),
    CHECK (due_date > period),
    CHECK ((status = 'PAID') = (paid_at IS NOT NULL)),
    CHECK ((status = 'PAID') = (paid_by IS NOT NULL))
);

-- Операция по балансу журнал(). Знак суммы и ссылка на источник
-- зависят от типа операции.
CREATE TABLE balance_operation (
    id              BIGSERIAL PRIMARY KEY,
    resident_id     INTEGER         NOT NULL REFERENCES resident (id),
    op_type         balance_op_type NOT NULL,
    amount          NUMERIC(12,2)   NOT NULL CHECK (amount <> 0),
    balance_after   NUMERIC(12,2)   NOT NULL,
    payment_id      INTEGER UNIQUE  REFERENCES payment (id),
    rent_charge_id  INTEGER UNIQUE  REFERENCES rent_charge (id),
    request_id      INTEGER         REFERENCES service_request (id),
    punishment_id   INTEGER UNIQUE  REFERENCES punishment (id),
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CHECK (op_type <> 'TOPUP'      OR (amount > 0 AND payment_id     IS NOT NULL)),
    CHECK (op_type <> 'RENT'       OR (amount < 0 AND rent_charge_id IS NOT NULL)),
    CHECK (op_type <> 'FEE'        OR (amount < 0 AND request_id     IS NOT NULL)),
    CHECK (op_type <> 'FEE_REFUND' OR (amount > 0 AND request_id     IS NOT NULL)),
    CHECK (op_type <> 'FINE'       OR (amount < 0 AND punishment_id  IS NOT NULL))
);

-- Правило изменения ИЛ фиксированный( перечень –RL01RL07)
CREATE TABLE loyalty_rule (
    code         CHAR(4)      PRIMARY KEY CHECK (code ~ '^RL[0-9]{2}$'),
    description  VARCHAR(200) NOT NULL,
    delta        SMALLINT     NOT NULL CHECK (delta <> 0)
);

-- Изменение ИЛ журнал(); delta_applied — изменение с учётом границ 0..100
CREATE TABLE loyalty_change (
```

```
    id              BIGSERIAL PRIMARY KEY,
    resident_id     INTEGER     NOT NULL REFERENCES resident (id),
    rule_code       CHAR(4)     NOT NULL REFERENCES loyalty_rule (code),
    delta_applied   SMALLINT    NOT NULL,
    value_after     SMALLINT    NOT NULL CHECK (value_after BETWEEN 0 AND 100),
    denunciation_id INTEGER     REFERENCES denunciation (id),
    punishment_id   INTEGER     REFERENCES punishment (id),
    rent_charge_id  INTEGER     REFERENCES rent_charge (id),
    request_id      INTEGER     REFERENCES service_request (id),
    comment         TEXT,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Заметка управляющего в досье жильца
CREATE TABLE dossier_note (
    id          SERIAL PRIMARY KEY,
    resident_id INTEGER     NOT NULL REFERENCES resident (id) ON DELETE CASCADE,
    author_id   INTEGER     NOT NULL REFERENCES account (id),
    text        TEXT        NOT NULL CHECK (btrim(text) <> ''),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Уведомление пользователю
CREATE TABLE notification (
    id          BIGSERIAL PRIMARY KEY,
    account_id  INTEGER     NOT NULL REFERENCES account (id) ON DELETE CASCADE,
    message     TEXT        NOT NULL,
    is_read     BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Справочные данные ограничение( 4 SRS): без них не работают внешние ключи
-- заявок и журнала ИЛ и триггер loyalty_change_apply.
INSERT INTO service (code, name, description, min_loyalty, fee, result_kind, needs_period) VALUES
 ('SV01', 'Бытовой␣ремонт',                          'Ремонт␣сантехники,␣электрики␣и␣иного␣
    оборудования␣квартиры.', 0,   0,   'WORK_MARK',       FALSE),
 ('SV02', 'Справка␣о␣проживании',                    'Справка,␣подтверждающая␣проживание␣в␣доме.',
                  20,  50,  'DOCUMENT',        FALSE),
 ('SV03', 'Продовольственный␣талон',                 'Талон␣на␣получение␣продуктов␣на␣указанный␣
    период.',           30,  50,  'TICKET',          TRUE),
 ('SV04', 'Разрешение␣на␣приём␣гостей',              'Разрешение␣принимать␣гостей␣в␣указанные␣даты.',
                  40,  100, 'PERMIT',          TRUE),
 ('SV05', 'Разрешение␣на␣выход␣в␣комендантский␣час', 'Разрешение␣находиться␣вне␣дома␣в␣комендантский␣
    час.',          50,  100, 'PERMIT',          TRUE),
 ('SV06', 'Замена␣мебели␣и␣бытовой␣техники',        'Замена␣мебели␣или␣бытовой␣техники␣в␣квартире.',
                  60,  300, 'WORK_MARK',       FALSE),
 ('SV07', 'Переселение␣в␣другую␣квартиру',           'Переселение␣в␣квартиру␣дома␣со␣свободным␣
    местом.',            70,  300, 'RELOCATION',      FALSE),
 ('SV08', 'Разрешение␣на␣выезд␣из␣города',           'Разрешение␣покинуть␣город␣на␣указанный␣период.'
    ,               80,  300, 'PERMIT',          TRUE),
 ('SV09', 'Снятие␣предупреждения␣из␣досье',         'Снятие␣последнего␣действующего␣предупреждения.'
    ,               85,  500, 'WARNING_REMOVAL', FALSE);

INSERT INTO loyalty_rule (code, description, delta) VALUES
 ('RL01', 'Донос␣жильца␣подтверждён␣кроме(␣анонимного)',                 5),
 ('RL02', 'Донос␣жильца␣признан␣ложным␣кроме(␣анонимного)',            -10),
 ('RL03', 'Квартплата␣оплачена␣в␣течение␣10␣дней␣с␣даты␣начисления',     1),
 ('RL04', 'Квартплата␣не␣оплачена␣в␣течение␣10␣дней␣с␣даты␣начисления', -5),
 ('RL05', 'Жильцу␣вынесено␣предупреждение',                            -10),
 ('RL06', 'Жильцу␣назначен␣штраф',                                     -15),
 ('RL07', 'Выполнена␣услуга␣SV09',                                      10);

COMMIT;
```

02_triggers.sql

```
-- ================================================================
-- ИС Бдительность«». Этап 2.
-- 02_triggers.sql — триггеры целостности данных.
--
-- Триггеры защищают только данные: журналы, согласованность баланса и ИЛ
-- с журналами, вместимость квартиры, модели состояний и историю заявок.
-- Бизнесправила– условия( подачи заявок, правила ИЛ, уведомления, права)
```

```
-- реализует серверная часть на Spring.
--
-- Коды ошибок (SQLSTATE), по которым Spring выбирает ответ HTTP:
--   BH404 — объект не найден;
--   BH409 — действие недопустимо в текущем состоянии;
--   BH422 — не выполнено условие нет( средств, неверный код, неподходящий донос).
-- Нарушения ограничений таблиц имеют стандартные коды:
--   23505 — UNIQUE, 23514 — CHECK, 23503 — внешний ключ.
-- ================================================================

BEGIN;

-- 1. Журналы нельзя изменять и удалять (S008)
CREATE FUNCTION trg_forbid_modification() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'Записи␣таблицы␣%␣не␣изменяются␣и␣не␣удаляются', TG_TABLE_NAME
        USING ERRCODE = 'BH409';
END;
$$;

CREATE TRIGGER balance_operation_immutable BEFORE UPDATE OR DELETE ON balance_operation
    FOR EACH ROW EXECUTE FUNCTION trg_forbid_modification();
CREATE TRIGGER loyalty_change_immutable BEFORE UPDATE OR DELETE ON loyalty_change
    FOR EACH ROW EXECUTE FUNCTION trg_forbid_modification();
CREATE TRIGGER request_event_immutable BEFORE UPDATE OR DELETE ON request_event
    FOR EACH ROW EXECUTE FUNCTION trg_forbid_modification();

-- 2. Операция по балансу изменяет баланс жильца и запоминает остаток.
--    В минус баланс уходит только изза- штрафа (S004).
CREATE FUNCTION trg_balance_operation_apply() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE
    v_balance NUMERIC(12,2);
BEGIN
    SELECT balance INTO v_balance FROM resident WHERE id = NEW.resident_id FOR UPDATE;
    IF NEW.amount < 0 AND NEW.op_type <> 'FINE' AND v_balance + NEW.amount < 0 THEN
        RAISE EXCEPTION 'Недостаточно␣средств:␣на␣балансе␣%␣руб.,␣требуется␣%␣руб.', v_balance, -NEW.
    amount
            USING ERRCODE = 'BH422';
    END IF;
    NEW.balance_after := v_balance + NEW.amount;
    UPDATE resident SET balance = NEW.balance_after WHERE id = NEW.resident_id;
    RETURN NEW;
END;
$$;

CREATE TRIGGER balance_operation_apply BEFORE INSERT ON balance_operation
    FOR EACH ROW EXECUTE FUNCTION trg_balance_operation_apply();

-- 3. Изменение ИЛ: величина берётся из правила, ИЛ остаётся в 0..100 (S005)
CREATE FUNCTION trg_loyalty_change_apply() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE
    v_current SMALLINT;
BEGIN
    SELECT loyalty_index INTO v_current FROM resident WHERE id = NEW.resident_id FOR UPDATE;
    NEW.value_after := LEAST(100, GREATEST(0,
        v_current + (SELECT delta FROM loyalty_rule WHERE code = NEW.rule_code)));
    NEW.delta_applied := NEW.value_after - v_current;
    UPDATE resident SET loyalty_index = NEW.value_after WHERE id = NEW.resident_id;
    RETURN NEW;
END;
$$;

CREATE TRIGGER loyalty_change_apply BEFORE INSERT ON loyalty_change
    FOR EACH ROW EXECUTE FUNCTION trg_loyalty_change_apply();

-- 4. Жилец:
--    - в квартире живут не более 10 жильцов ограничение( 7 SRS); строка
--      квартиры блокируется, чтобы одновременные заселения шли по очереди;
--    - новый жилец начинает с баланса 0 и ИЛ 50 (C002), дальше баланс и ИЛ
--      меняют только триггеры журналов они( работают на глубине >= 2).
CREATE FUNCTION trg_resident_guard() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
```

```
    IF NEW.status = 'ACTIVE' AND (TG_OP = 'INSERT' OR NEW.apartment_id <> OLD.apartment_id
                                  OR OLD.status <> 'ACTIVE') THEN
        PERFORM 1 FROM apartment WHERE id = NEW.apartment_id FOR UPDATE;
        IF (SELECT count(*) FROM resident
            WHERE apartment_id = NEW.apartment_id AND status = 'ACTIVE' AND id <> NEW.id) >= 10 THEN
            RAISE EXCEPTION 'В␣квартире␣уже␣проживает␣10␣жильцов' USING ERRCODE = 'BH409';
        END IF;
    END IF;

    IF TG_OP = 'INSERT' AND (NEW.balance <> 0 OR NEW.loyalty_index <> 50) THEN
        RAISE EXCEPTION 'Новый␣жилец␣создаётся␣с␣балансом␣0␣и␣ИЛ␣50' USING ERRCODE = 'BH409';
    END IF;
    IF TG_OP = 'UPDATE' AND pg_trigger_depth() < 2
       AND (NEW.balance <> OLD.balance OR NEW.loyalty_index <> OLD.loyalty_index) THEN
        RAISE EXCEPTION 'Баланс␣и␣ИЛ␣изменяются␣только␣через␣журналы' USING ERRCODE = 'BH409';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER resident_guard BEFORE INSERT OR UPDATE ON resident
    FOR EACH ROW EXECUTE FUNCTION trg_resident_guard();

-- 5. Модель состояний заявки (S009): заявка создаётся в статусе SUBMITTED,
--    далее — только допустимые переходы.
CREATE FUNCTION trg_request_status() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' AND NEW.status <> 'SUBMITTED' THEN
        RAISE EXCEPTION 'Заявка␣создаётся␣в␣статусе␣SUBMITTED' USING ERRCODE = 'BH409';
    END IF;
    IF TG_OP = 'UPDATE' AND NEW.status <> OLD.status AND NOT (
           (OLD.status = 'SUBMITTED'           AND NEW.status IN ('IN_PROGRESS', 'REJECTED', '
    CANCELLED'))
        OR (OLD.status = 'IN_PROGRESS'         AND NEW.status IN ('NEEDS_CLARIFICATION', 'COMPLETED'
    , 'REJECTED'))
        OR (OLD.status = 'NEEDS_CLARIFICATION' AND NEW.status = 'IN_PROGRESS')) THEN
        RAISE EXCEPTION 'Недопустимый␣переход␣статуса␣заявки:␣%␣→␣%', OLD.status, NEW.status
            USING ERRCODE = 'BH409';
    END IF;
    NEW.updated_at := now();
    RETURN NEW;
END;
$$;

CREATE TRIGGER request_status BEFORE INSERT OR UPDATE ON service_request
    FOR EACH ROW EXECUTE FUNCTION trg_request_status();

-- 6. История заявки (S008): подача и каждая смена статуса записываются
--    с исполнителем и комментарием из status_changed_by и status_comment.
CREATE FUNCTION trg_request_history() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO request_event (request_id, to_status, actor_id, comment)
        VALUES (NEW.id, NEW.status, NEW.status_changed_by, NEW.status_comment);
    ELSIF NEW.status <> OLD.status THEN
        INSERT INTO request_event (request_id, from_status, to_status, actor_id, comment)
        VALUES (NEW.id, OLD.status, NEW.status, NEW.status_changed_by, NEW.status_comment);
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER request_history AFTER INSERT OR UPDATE ON service_request
    FOR EACH ROW EXECUTE FUNCTION trg_request_history();

-- 7. Модель состояний доноса (S009): донос создаётся в статусе SUBMITTED,
--    решение принимается один раз.
CREATE FUNCTION trg_denunciation_status() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' AND NEW.status <> 'SUBMITTED' THEN
        RAISE EXCEPTION 'Донос␣создаётся␣в␣статусе␣SUBMITTED' USING ERRCODE = 'BH409';
    END IF;
    IF TG_OP = 'UPDATE' AND NEW.status <> OLD.status AND OLD.status <> 'SUBMITTED' THEN
```

```
        RAISE EXCEPTION 'Донос␣уже␣рассмотрен␣статус(␣%)', OLD.status USING ERRCODE = 'BH409';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER denunciation_status BEFORE INSERT OR UPDATE ON denunciation
    FOR EACH ROW EXECUTE FUNCTION trg_denunciation_status();

-- 8. Основание наказания связь( M:N) — подтверждённый донос на того же
--    жильца, которому назначено наказание (C013).
CREATE FUNCTION trg_punishment_basis_check() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM denunciation d JOIN punishment p ON p.id = NEW.punishment_id
                   WHERE d.id = NEW.denunciation_id AND d.status = 'CONFIRMED'
                     AND d.suspect_id = p.resident_id) THEN
        RAISE EXCEPTION 'Донос␣%␣не␣подтверждён␣или␣подан␣не␣на␣наказываемого␣жильца', NEW.
    denunciation_id
            USING ERRCODE = 'BH422';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER punishment_basis_check BEFORE INSERT OR UPDATE ON punishment_basis
    FOR EACH ROW EXECUTE FUNCTION trg_punishment_basis_check();

COMMIT;
```

03_functions.sql

```
-- ======================================================================
-- ИС Бдительность«». Этап 2.
-- 03_functions.sql — функции для критически важных запросов
-- и представления.
--
-- Бизнеслогику– условия( подачи заявок, правила ИЛ, уведомления, права)
-- реализует серверная часть на Spring, простые чтения и изменения она
-- выполняет через ORM. Функции нужны там, где операция должна быть
-- атомарной или выполняться одним запросом над многими строками:
-- активация по коду, зачисление и списание денег, ежемесячные расчёты,
-- очередь заявок и досье.
-- ======================================================================

BEGIN;

-- ======================================================================
-- Представления
-- ======================================================================

-- Доносы для управляющего: автор анонимного доноса не раскрывается (N005)
CREATE VIEW v_denunciation_for_manager AS
SELECT d.id,
       CASE WHEN d.is_anonymous THEN NULL ELSE d.author_id END AS author_id,
       d.suspect_id, d.description, d.incident_date, d.is_anonymous,
       d.status, d.review_comment, d.created_at, d.reviewed_at
FROM denunciation d;

-- История ИЛ для жильца: ссылки на доносы не выдаются (U004, N005)
CREATE VIEW v_loyalty_history AS
SELECT lc.id, lc.resident_id, lc.rule_code, lr.description AS rule_description,
       lc.delta_applied, lc.value_after, lc.comment, lc.created_at
FROM loyalty_change lc JOIN loyalty_rule lr ON lr.code = lc.rule_code;

-- Квартиры с числом проживающих и их списком (C001)
CREATE VIEW v_apartment_occupancy AS
SELECT a.id, a.number, a.floor, a.area, a.rent_tariff,
       count(r.id)::INTEGER AS resident_count,
       string_agg(r.last_name || '␣' || r.first_name, ',␣' ORDER BY r.last_name) AS residents
FROM apartment a
LEFT JOIN resident r ON r.apartment_id = a.id AND r.status = 'ACTIVE'
GROUP BY a.id;
```

```
-- ========================================================================
-- Учётная запись
-- ========================================================================

-- UC01. Активировать учётную запись по коду приглашения.
-- Код помечается использованным и привязывается к новой учётной записи
-- в одной транзакции, поэтому один код нельзя активировать дважды даже
-- при одновременных запросах. Пароль хеширует приложение (bcrypt, N004).
CREATE FUNCTION activate_account(p_code CHAR(8), p_login VARCHAR, p_password_hash VARCHAR)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_resident INTEGER;
    v_account  INTEGER;
BEGIN
    UPDATE invitation SET used_at = now()
    WHERE code = upper(p_code) AND used_at IS NULL
    RETURNING resident_id INTO v_resident;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Код␣приглашения␣неверен␣или␣уже␣использован' USING ERRCODE = 'BH422';
    END IF;

    INSERT INTO account (login, password_hash, role)
    VALUES (p_login, p_password_hash, 'RESIDENT')
    RETURNING id INTO v_account;

    UPDATE resident SET account_id = v_account WHERE id = v_resident AND status = 'ACTIVE';
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Жилец␣выселен,␣активация␣невозможна' USING ERRCODE = 'BH409';
    END IF;
    RETURN v_account;
END;
$$;

-- ========================================================================
-- Баланс и квартплата
-- ========================================================================

-- UC04. Применить статус платежа, полученный из ЮKassa ограничение( 6 SRS).
-- Обрабатывается только платёж в статусе PENDING, поэтому повторный вызов
-- ничего не меняет и деньги не зачисляются дважды (S003, N007).
-- Возвращает TRUE, если средства зачислены именно этим вызовом.
CREATE FUNCTION apply_payment_status(p_yookassa_id VARCHAR, p_status payment_status)
RETURNS BOOLEAN
LANGUAGE plpgsql AS $$
DECLARE
    v_pay payment%ROWTYPE;
BEGIN
    IF p_status = 'PENDING' THEN
        RETURN FALSE;
    END IF;

    UPDATE payment
    SET status = p_status,
        confirmed_at = CASE WHEN p_status = 'SUCCEEDED' THEN now() END
    WHERE yookassa_payment_id = p_yookassa_id AND status = 'PENDING'
    RETURNING * INTO v_pay;

    IF NOT FOUND THEN
        IF NOT EXISTS (SELECT 1 FROM payment WHERE yookassa_payment_id = p_yookassa_id) THEN
            RAISE EXCEPTION 'Платёж␣%␣не␣найден', p_yookassa_id USING ERRCODE = 'BH404';
        END IF;
        RETURN FALSE;   -- платёж уже обработан
    END IF;

    IF p_status = 'SUCCEEDED' THEN
        INSERT INTO balance_operation (resident_id, op_type, amount, payment_id)
        VALUES (v_pay.resident_id, 'TOPUP', v_pay.amount, v_pay.id);
    END IF;
    RETURN p_status = 'SUCCEEDED';
END;
$$;

-- UC06. Оплатить начисление квартплаты с баланса жильца. Начисление
-- выставлено квартире, оплатить его может любой её жилец, но только один
```

```
-- раз: строка начисления блокируется, и второй платёж получит ошибку.
-- Отметка об оплате и списание выполняются вместе; при недостатке средств
-- триггер журнала отменяет всю операцию. Возвращает TRUE, если оплата
-- сделана в срок по( этому признаку приложение применяет правило RL03).
CREATE FUNCTION pay_rent(p_resident_id INTEGER, p_charge_id INTEGER)
RETURNS BOOLEAN
LANGUAGE plpgsql AS $$
DECLARE
    v_rc rent_charge%ROWTYPE;
BEGIN
    UPDATE rent_charge SET status = 'PAID', paid_at = now(), paid_by = p_resident_id
    WHERE id = p_charge_id AND status = 'UNPAID'
      AND apartment_id = (SELECT apartment_id FROM resident
                          WHERE id = p_resident_id AND status = 'ACTIVE')
    RETURNING * INTO v_rc;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Начисление␣%␣не␣относится␣к␣квартире␣жильца␣%␣или␣уже␣оплачено',
            p_charge_id, p_resident_id
            USING ERRCODE = 'BH409';
    END IF;

    IF v_rc.amount > 0 THEN
        INSERT INTO balance_operation (resident_id, op_type, amount, rent_charge_id)
        VALUES (p_resident_id, 'RENT', -v_rc.amount, v_rc.id);
    END IF;
    RETURN v_rc.paid_at::date <= v_rc.due_date;
END;
$$;

-- S001. Начислить квартплату за месяц одним запросом: одно начисление
-- в размере тарифа на каждую квартиру, где живёт хотя бы один жилец,
-- заселённый до конца месяца. Повторный запуск за тот же месяц дублей
-- не создаёт. Возвращает число новых начислений.
CREATE FUNCTION accrue_rent(p_period DATE)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_period DATE := date_trunc('month', p_period)::date;
    v_count  INTEGER;
BEGIN
    INSERT INTO rent_charge (apartment_id, period, amount, due_date)
    SELECT a.id, v_period, a.rent_tariff, v_period + 10
    FROM apartment a
    WHERE EXISTS (SELECT 1 FROM resident r
                  WHERE r.apartment_id = a.id AND r.status = 'ACTIVE'
                    AND r.settled_at < v_period + INTERVAL '1␣month')
    ON CONFLICT DO NOTHING;
    GET DIAGNOSTICS v_count = ROW_COUNT;
    RETURN v_count;
END;
$$;

-- S002. Отметить начисления, не оплаченные в срок, и вернуть жильцов
-- этих квартир с активированной учётной записью: приложение применяет
-- к каждому правило RL04 и отправляет уведомления.
CREATE FUNCTION register_overdue_rent(p_today DATE DEFAULT current_date)
RETURNS TABLE (charge_id INTEGER, resident_id INTEGER, period DATE)
LANGUAGE sql AS $$
    WITH marked AS (
        UPDATE rent_charge rc SET overdue_registered = TRUE
        WHERE rc.status = 'UNPAID' AND NOT rc.overdue_registered AND rc.due_date < p_today
        RETURNING rc.id, rc.apartment_id, rc.period)
    SELECT m.id, r.id, m.period
    FROM marked m JOIN resident r ON r.apartment_id = m.apartment_id
    WHERE r.status = 'ACTIVE' AND r.account_id IS NOT NULL;
$$;

-- ================================================================
-- Выборки для управляющего
-- ================================================================

-- UC17. Очередь необработанных заявок: по убыванию ИЛ заявителя,
-- затем по дате подачи (C006).
CREATE FUNCTION request_queue()
RETURNS TABLE (request_id INTEGER, created_at TIMESTAMPTZ, status request_status,
               service_name VARCHAR, resident_id INTEGER, resident_name TEXT,
```

```
             apartment_number INTEGER, loyalty_index SMALLINT)
LANGUAGE sql STABLE AS $$
    SELECT sr.id, sr.created_at, sr.status, s.name,
           r.id, r.last_name || ' ' || r.first_name, a.number, r.loyalty_index
    FROM service_request sr
    JOIN resident  r ON r.id = sr.resident_id
    JOIN service   s ON s.id = sr.service_id
    JOIN apartment a ON a.id = r.apartment_id
    WHERE sr.status IN ('SUBMITTED', 'IN_PROGRESS', 'NEEDS_CLARIFICATION')
      AND r.status = 'ACTIVE'
    ORDER BY r.loyalty_index DESC, sr.created_at, sr.id;
$$;

-- UC15. Досье жильца одним документом JSON: анкета, доносы на жильца
-- и поданные им, наказания, история ИЛ, заявки, операции, заметки (C017).
-- Авторы анонимных доносов не выдаются (N005).
CREATE FUNCTION resident_dossier(p_resident_id INTEGER)
RETURNS JSONB
LANGUAGE sql STABLE AS $$
    SELECT jsonb_build_object(
        'resident', (SELECT to_jsonb(x) FROM (
            SELECT r.id, r.last_name, r.first_name, r.patronymic, r.birth_date,
                   r.phone, a.number AS apartment, r.loyalty_index, r.balance,
                   r.status, r.settled_at, r.evicted_at, r.eviction_reason
            FROM resident r JOIN apartment a ON a.id = r.apartment_id
            WHERE r.id = p_resident_id) x),
        'denunciations_against', coalesce((SELECT jsonb_agg(to_jsonb(d) ORDER BY d.created_at DESC,
    d.id DESC)
            FROM v_denunciation_for_manager d WHERE d.suspect_id = p_resident_id), '[]'),
        'denunciations_filed', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', d.id, 'suspect_id', d.suspect_id, 'status', d.status,
                'created_at', d.created_at) ORDER BY d.created_at DESC, d.id DESC)
            FROM denunciation d WHERE d.author_id = p_resident_id AND NOT d.is_anonymous), '[]'),
        'punishments', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', p.id, 'type', p.type, 'amount', p.amount, 'reason', p.reason,
                'created_at', p.created_at, 'is_revoked', p.is_revoked,
                'denunciations', (SELECT coalesce(jsonb_agg(pb.denunciation_id), '[]')
                                  FROM punishment_basis pb WHERE pb.punishment_id = p.id))
                ORDER BY p.created_at DESC, p.id DESC)
            FROM punishment p WHERE p.resident_id = p_resident_id), '[]'),
        'loyalty_history', coalesce((SELECT jsonb_agg(to_jsonb(h) ORDER BY h.created_at DESC, h.id
    DESC)
            FROM v_loyalty_history h WHERE h.resident_id = p_resident_id), '[]'),
        'requests', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', sr.id, 'service', s.name, 'status', sr.status,
                'created_at', sr.created_at) ORDER BY sr.created_at DESC, sr.id DESC)
            FROM service_request sr JOIN service s ON s.id = sr.service_id
            WHERE sr.resident_id = p_resident_id), '[]'),
        'operations', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', bo.id, 'type', bo.op_type, 'amount', bo.amount,
                'balance_after', bo.balance_after, 'created_at', bo.created_at)
                ORDER BY bo.created_at DESC, bo.id DESC)
            FROM balance_operation bo WHERE bo.resident_id = p_resident_id), '[]'),
        'notes', coalesce((SELECT jsonb_agg(jsonb_build_object(
                'id', n.id, 'text', n.text, 'created_at', n.created_at)
                ORDER BY n.created_at DESC, n.id DESC)
            FROM dossier_note n WHERE n.resident_id = p_resident_id), '[]')
    );
$$;

COMMIT;
```

04_indexes.sql

```
-- ======================================================================
-- ИС Бдительность«». Этап 2.
-- 04_indexes.sql — индексы для самых частых запросов прецедентов.
-- Индексы первичных ключей и ограничений UNIQUE создаются автоматически
-- (01_create.sql); здесь созданы индексы, которые PostgreSQL сам не создаёт.
-- ======================================================================

BEGIN;
```

```
-- I1. UC17 Посмотреть« список заявок»: очередь необработанных заявок.
-- Частичный индекс содержит только активные заявки, доля которых мала.
CREATE INDEX idx_request_active ON service_request (status, created_at)
    WHERE status IN ('SUBMITTED', 'IN_PROGRESS', 'NEEDS_CLARIFICATION');

-- I2. UC10 Мои« заявки»: заявки жильца, новые сначала.
CREATE INDEX idx_request_resident ON service_request (resident_id, created_at DESC);

-- I3. UC10: история обработки заявки.
CREATE INDEX idx_request_event_request ON request_event (request_id, created_at);

-- I4. UC22 Рассмотреть« донос»: очередь нерассмотренных доносов.
CREATE INDEX idx_denunciation_pending ON denunciation (created_at)
    WHERE status = 'SUBMITTED';

-- I5. UC05 Просмотреть« баланс и историю операций».
CREATE INDEX idx_balance_operation_resident ON balance_operation (resident_id, created_at DESC);

-- I6. Непрочитанные уведомления на главной странице кабинета.
CREATE INDEX idx_notification_unread ON notification (account_id, created_at DESC)
    WHERE NOT is_read;

COMMIT;
```